

# Python Tuples Complete Guide

Core Concepts, Immutability, Hashability, and Interview Questions

## 1. Understanding Core Tuple Concepts

### The Locked List (Immutable)

Think of a Tuple as a **locked list**. Once you put items inside and close the parentheses, you throw away the key.

- Just like Strings, Tuples cannot be changed in place. You cannot add, remove, or modify elements directly. This makes them incredibly fast and memory-efficient.

### Key Characteristics

- **Ordered:** Like Lists, the items stay in the exact order you put them in, and you can access them using an index (starting at `0`).
- **Tuple Unpacking:** This is a Python superpower. You can instantly extract all items from a tuple and assign them to variables in a single line: `name, age, degree = student`.
- **The Single-Item Comma Rule:** If you write `x = (5)`, Python thinks the parentheses are just for math and makes `x` an integer. To tell Python it is a tuple, you **must** include a trailing comma: `x = (5,)`.

## 2. Tuple Methods Code Examples

Because tuples cannot be changed, they don't have methods like `append()` or `remove()`. They only have two methods, both used for reading data.

## 1. count(x)

Counts exactly how many times a specific value appears in the tuple.

```
# A tuple recording the results of coin flips
coin_flips = ("Heads", "Tails", "Heads", "Heads", "Tails")

# Count how many times "Heads" appears
heads_count = coin_flips.count("Heads")

print("You flipped Heads", heads_count, "times.")
# Output: You flipped Heads 3 times.
```

## 2. index(x)

Searches for a specific value and returns the **index number** of its very first occurrence.

```
# A tuple of fixed RGB color values
colors = ("Red", "Green", "Blue", "Yellow")

# Find the position of "Blue"
blue_position = colors.index("Blue")

print("The color Blue is at index:", blue_position)
# Output: The color Blue is at index: 2
```

## 3. The "CRUD" Workaround

---

To "Update" or "Delete" an item in a tuple, you have to use a workaround. Because they are locked (immutable), you literally have to slice the tuple apart, inject your new item (as another tuple), and glue it all back together into a **brand-new tuple**.

```
# Original Tuple
numbers = (1, 2, 3)

# We want to change the '2' to a '20'.
# 1. Take everything before the 2: numbers[:1]
# 2. Add our new tuple: (20,)
# 3. Take everything after the 2: numbers[2:]

numbers = numbers[:1] + (20,) + numbers[2:]
print(numbers)
# Output: (1, 20, 3)
```

*Note: If you find yourself doing this often in real-world programming, you should probably just be using a List!*

## 4. Interview Questions Expanded

---

### 1. What does "Hashable" mean?

For an object to be a "key" in a Python Dictionary, it must be hashable—meaning its value can never change.

- **Lists** can change (mutable), so they are **not hashable**. You cannot use a list as a dictionary key.
- **Tuples** cannot change (immutable), so they **are hashable**. You *can* use a tuple as a dictionary key (for example, using (latitude, longitude) coordinates as a key to store a city name).

### 2. The "List inside a Tuple" Trap

If you put a List *inside* a Tuple, like `my_tuple = (1, 2, [3, 4])`, the tuple itself is locked, but the list inside it can still be modified! Because the list inside can change, the entire tuple loses its "hashable" status and can no longer be used as a dictionary key.